

SarvamAI Playground

On-Device AI Developer Portal

Frontend Intern Assignment — Submission

Candidate	Jayanth
Role	Frontend Intern
GitHub	github.com/jayanthct/SarvamAI-Playground
Live Demo	sarvam-ai-playground.vercel.app
Figma Design	View Figma File
Video Walkthrough	Watch 3-min Walkthrough
Submitted	19 May 2026

Contents

1	Project Links	3
1.1	GitHub Repository	3
1.2	Live Deployment	3
1.3	Figma Design File	3
1.4	3-Minute Video Walkthrough	3
2	Architecture Decisions	4
2.1	Tech Stack	4
2.2	Project Structure	4
2.3	LLMResponse Component	4
2.4	Models Used	5
2.5	Streaming Architecture	5
3	Diffing Algorithm	6
3.1	Algorithm: Word-level LCS via Dynamic Programming	6
3.2	Worked Example	6
3.3	TypeScript Implementation	6
4	Time Complexity Analysis	8
4.1	Optimisation Headroom	8
5	Why LCS over Alternatives	9
5.1	Myers Diff — Not Selected	9
5.2	Why LCS Was Selected	9
6	Accessibility Considerations	10
7	Error Handling Strategy	11
7.1	Error Scenarios	11
7.2	Implementation Details	11
8	Part B — Bug Report	12
8.1	Description	12
8.2	Steps to Reproduce	12
8.3	Expected Behaviour	12
8.4	Actual Behaviour	12
8.5	Root Cause	12
8.6	Fix Applied	12

1. Project Links

1.1 GitHub Repository

github.com/jayanthct/SarvamAI-Playground

Contains full source code, component structure, streaming logic, diff algorithm, and this submission document.

1.2 Live Deployment

sarvam-ai-playground.vercel.app

Fully functional in browser.

1.3 Figma Design File

[View Full Design on Figma](#)

Includes all screens: wireframes, assets, components typography, color-schema.

1.4 3-Minute Video Walkthrough

[Watch Walkthrough Video](#)

Covers: multi-modal input toggle, live streaming with metrics, error handling scenarios, diff view with token highlights, and keyboard navigation demo.

2. Architecture Decisions

The application is structured around two primary pages — the Inference Playground (Part A) and the Diff View (Part B) — sharing a common input context via the React Context API.

2.1 Tech Stack

Layer	Choice	Reason
Framework	React 18 + TypeScript	Component model suits the playground/diff split naturally
Build Tool	Vite 5	Fast HMR, first-class TypeScript support
Styling	Tailwind CSS	Utility-first, consistent spacing and colour tokens
Streaming	Fetch API + ReadableStream	Native browser APIs, no third-party dependency
Speech	Web Speech API	Built-in browser, no SDK required
State	React Context + useState	Lightweight, sufficient for this feature scope
Fonts	Matter + Season	Sarvam.ai design system fonts
Deployment	Vercel	Auto-deploy from GitHub, SPA rewrite via <code>vercel.json</code>

2.2 Project Structure

Path	Responsibility
<code>src/lib/mockResponse.ts</code>	SSE chunk builders and mock token arrays for two model versions
<code>src/lib/mockStream.ts</code>	ReadableStream factory — drop-in replacement for real <code>fetch()</code>
<code>src/lib/inferenceStream.ts</code>	Core streaming loop, SSE parser, metrics calculation
<code>src/lib/diffAlgorithm.ts</code>	LCS dynamic programming diff — exports <code>DiffToken</code> type and <code>lcsDiff()</code>
<code>src/components/Textarea/</code> <code>src/components/LLMResponse/</code>	Multi-modal input with Web Speech API integration Renders streamed output for one model; handles diff token display per panel
<code>src/context/ChatContext/</code>	Shared state: prompt, submission flag, <code>showDiff</code> , <code>diffResult</code>
<code>src/pages/Chat/</code> <code>src/pages/Diff/</code>	Part A — Inference playground with live metrics Part B — Side-by-side diff view, two <code>LLMResponse</code> panels

2.3 LLMResponse Component

The `LLMResponse` component is the core rendering unit for both Part A and Part B. It accepts a `modelName`, streamed `chunks` string, `delay`, `tokenCount`, and `tokensPerSecond` as props.

It operates in two modes depending on `showDiff` from `ChatContext`:

- **Streaming mode** — renders the raw `chunks` string as it accumulates, with a “Generating...” shimmer state during the initial `delay` period.
- **Diff mode** — maps over `diffResult.tokens` from context and renders each `DiffToken` with colour coding. The left panel (Gemma) hides **added** tokens; the right panel (GPT-OSS 20B) hides **removed** tokens — so each panel only shows its own output with the opposing changes highlighted.

Token Type	Panel	Rendering
same	Both	Plain text, no highlight
added	Right only	Green text, green tinted background
removed	Left only	Red text, red tinted background, strikethrough

Metrics (`tokenCount` and `tokensPerSecond`) are displayed as an overlay in the bottom-right corner of each panel, updating live on every streamed chunk.

2.4 Models Used

Two mock model versions are configured, each with a distinct token array in `mockResponse.ts`:

Model	Key	Icon
Gemma 2B	<code>gemma-2b</code>	Google Gemini logo (<code>gemini.svg</code>)
GPT-OSS 20B	<code>gptoss20b</code>	OpenAI logo (<code>openai.svg</code>)

Model identity is resolved by checking whether `modelName.toLowerCase().includes('gemma')` — no hardcoded conditionals outside of this single check.

2.5 Streaming Architecture

The streaming pipeline is designed so the mock and real API are interchangeable at a single line:

```
// Mock (current)
const response = mockFetch(modelName);

// Real API (swap-in, no other changes needed)
const response = await fetch('/api/inference', {
  method: 'POST',
  body: JSON.stringify({ prompt }),
  signal,
});
```

The SSE parser, metrics engine, `AbortController`, and error handling all operate identically regardless of the source. Swapping to production requires changing one line only.

3. Diffing Algorithm

3.1 Algorithm: Word-level LCS via Dynamic Programming

Both model outputs are split into word token arrays. A 2D DP table of size $(m+1) \times (n+1)$ is constructed where m and n are the word counts of each output. Every cell $dp[i][j]$ stores the length of the longest common subsequence between the first i words of output A and the first j words of output B.

After the table is filled, it is backtracked from the bottom-right corner to classify every word as **same**, **added**, or **removed**. The result is an ordered token array used to render colour-coded highlights in the UI.

3.2 Worked Example

Input A	“the cat sat on the mat”
Input B	“the dog sat on the floor”
LCS found	[“the”, “sat”, “on”, “the”] → length 4

Backtrack result:

Word	Status	Reason
the	Same	Present in both outputs
cat	Removed	Only in A — not in LCS
dog	Added	Only in B — not in LCS
sat	Same	Present in both outputs
on	Same	Present in both outputs
the	Same	Present in both outputs
mat	Removed	Only in A — not in LCS
floor	Added	Only in B — not in LCS

3.3 TypeScript Implementation

```
export function lcsDiff(strA: string, strB: string): DiffResult {
  const wordsA = strA.trim().split(/\s+/);
  const wordsB = strB.trim().split(/\s+/);
  const m = wordsA.length;
  const n = wordsB.length;

  // Build DP table
  const dp: number[][] = Array.from(
    { length: m + 1 },
    () => Array(n + 1).fill(0)
  );

  for (let i = 1; i <= m; i++) {
    for (let j = 1; j <= n; j++) {
      if (wordsA[i - 1] === wordsB[j - 1]) {
        dp[i][j] = dp[i - 1][j - 1] + 1;
      } else {
        dp[i][j] = Math.max(dp[i - 1][j], dp[i][j - 1]);
      }
    }
  }
}
```

```
    }  
  }  
  
  // Backtrack  
  const tokens: DiffToken[] = [];  
  let i = m, j = n;  
  
  while (i > 0 || j > 0) {  
    if (i > 0 && j > 0 && wordsA[i-1] === wordsB[j-1]) {  
      tokens.unshift({ word: wordsA[i - 1], type: 'same' });  
      i--; j--;  
    } else if (j > 0 && (i === 0 || dp[i][j-1] >= dp[i-1][j])) {  
      tokens.unshift({ word: wordsB[j - 1], type: 'added' });  
      j--;  
    } else {  
      tokens.unshift({ word: wordsA[i - 1], type: 'removed' });  
      i--;  
    }  
  }  
}  
  
const added    = tokens.filter(t => t.type === 'added').length;  
const removed = tokens.filter(t => t.type === 'removed').length  
;  
return { tokens, added, removed };  
}
```

4. Time Complexity Analysis

Phase	Complexity	Reason
Split into words	$O(m + n)$	One pass through each string
Build DP table	$O(m \times n)$	Every cell filled exactly once
Backtrack	$O(m + n)$	One path from corner to origin
Count tokens	$O(m + n)$	Single filter pass over token array
Overall	$O(m \times n)$	DP table construction dominates

Space complexity: $O(m \times n)$ for storing the full DP table.

For typical on-device model outputs with `max_new_tokens: 512`, both m and n stay under 500 words, making the grid at most 250,000 cells — runs in under 1ms in the browser.

4.1 Optimisation Headroom

If outputs were very large, space could be reduced to $O(n)$ using a two-row rolling array since each row only depends on the row directly above it:

```
let prev = Array(n + 1).fill(0);
let curr = Array(n + 1).fill(0);

for (let i = 1; i <= m; i++) {
  for (let j = 1; j <= n; j++) {
    if (wordsA[i - 1] === wordsB[j - 1]) {
      curr[j] = prev[j - 1] + 1;
    } else {
      curr[j] = Math.max(prev[j], curr[j - 1]);
    }
  }
  prev = [...curr];
  curr.fill(0);
}
```

This reduces memory from 25 million cells to 10,000 cells for a 5000×5000 input. The full-table approach is retained in this implementation because backtracking requires the complete grid to recover the token label sequence. This optimisation is not needed at the text lengths this application targets.

5. Why LCS over Alternatives

5.1 Myers Diff — Not Selected

Myers diff is the algorithm behind `git diff`, optimised for line-level diffing of source code. It works by finding the shortest edit script between two sequences, operating on a diagonal edit graph.

Why not selected:

- Designed for line-level granularity. Applying it word-by-word adds implementation complexity with no benefit at this scale.
- Runs in $O(n \times d)$ where d is the number of differences. For AI outputs that diverge significantly this degrades toward $O(n^2)$ anyway — the same asymptotic bound as LCS.
- Navigating diagonal edit graph paths significantly increases implementation complexity for no practical gain on short AI output paragraphs.
- The assignment requires building the diff algorithm from scratch. Myers requires carefully tracking furthest-reaching D-paths on an edit graph, which is harder to verify correctness on than a DP table where every cell has a clear meaning.

5.2 Why LCS Was Selected

- Produces the exact token sequence — **same**, **added**, **removed** — directly from the backtrack step, ready for UI rendering without post-processing.
- Insertion and deletion aware — a word that shifts position is correctly identified as unchanged, unlike position-based comparison.
- $O(m \times n)$ is fast enough for all realistic AI output lengths in this application.
- Transparent and verifiable — every cell in the DP grid has a clear, testable meaning.
- No external libraries — fully self-contained in under 50 lines of TypeScript as required by the assignment.

6. Accessibility Considerations

The application meets WCAG AA standards. All interactive elements are keyboard navigable and screen-reader compatible.

Area	Implementation
Keyboard navigation	Tab to move between controls, Enter to submit, Escape to stop streaming
ARIA labels	All buttons have <code>aria-label</code> and <code>title</code> attributes
Live regions	Metrics use <code>aria-live="polite" + aria-atomic="false"</code> — screen readers announce token updates without interrupting speech
Error alerts	Error banners use <code>role="alert" + aria-live="assertive"</code> — immediately announced on failure
Audio waveform	Decorative bars are <code>aria-hidden="true"</code> — not announced to screen readers
Input mode toggle	<code>role="group"</code> on container, <code>aria-pressed</code> on each toggle button reflects current state
Focus rings	All interactive elements have <code>focus-visible:ring-2</code> — visible keyboard focus indicator
Colour contrast	All text combinations tested to WCAG AA — minimum 4.5:1 ratio on body text
Textarea	<code>aria-label="Prompt input", aria-multiline="true", aria-required="true"</code>

7. Error Handling Strategy

The core principle is: **never blank the screen, never lose partial output**. Every failure mode preserves what the user already received and displays a clear, specific error message.

7.1 Error Scenarios

Scenario	Detection	Response
Network drop	<code>fetch()</code> throws	Partial output preserved, error banner shown
Model timeout	No response within window	Timeout message shown, retry suggested
User interrupted	<code>AbortController.abort()</code>	Stream stops, partial output preserved
Malformed SSE chunk	<code>JSON.parse()</code> throws	Chunk silently skipped, stream continues
No readable body	<code>getReader()</code> returns null	Early return with error, no crash

7.2 Implementation Details

- `AbortController` is stored in a `useRef` — persists across renders, cleaned up on component unmount via the `useEffect` return function.
- Partial text is tracked in a local variable inside the streaming loop — avoids stale closure issues that would occur with `useState` inside an `async` loop.
- `reader.releaseLock()` is always called in the `finally` block — prevents stream resource leaks.
- `AbortError` is distinguished from other errors — user-initiated stops do not trigger the error UI.

8. Part B — Bug Report

Bug: Position-based comparison fails on shifted tokens

Component	DiffView — token comparison logic
Severity	High — produces incorrect diff results for most real-world model output pairs
Type	Logic error — incorrect algorithm selection

8.1 Description

A naive position-based comparison approach (comparing `word[i]` of output A directly against `word[i]` of output B) produces entirely incorrect results when any word is inserted or removed near the beginning of one output. Every subsequent word shifts index position and registers as changed even if it is identical in both outputs.

8.2 Steps to Reproduce

1. Set Model A output to: "cat sat on mat"
2. Set Model B output to: "big cat sat on mat"
3. Run the diff with a position-based comparator
4. Observe: `cat` shows as changed, `sat` shows as changed, `on` shows as changed, `mat` shows as removed — all incorrect

8.3 Expected Behaviour

Only "big" should be highlighted as added. All other words — `cat`, `sat`, `on`, `mat` — should render as unchanged.

8.4 Actual Behaviour

Every word after the insertion point is incorrectly flagged as changed, producing 4 false positives out of 5 total words.

8.5 Root Cause

The comparison algorithm assumes both outputs have words at corresponding indices. It has no concept of insertion or deletion shifting subsequent positions. This is a fundamental limitation of $O(n)$ position-based comparison — it cannot distinguish a shifted match from a genuine change.

8.6 Fix Applied

Replaced position-based comparison with a word-level LCS dynamic programming algorithm. LCS finds the longest sequence of words common to both outputs in order, correctly identifying unchanged words regardless of their position. The fix is implemented in `src/lib/diffAlgorithm.ts`.